
Requirements, Design and Tasks for raft-rx

FSM-based Raft Implementation

2026-05-10

Contents

Requisitos	5
Objetivo	5
Alcance funcional	5
Requisitos de arquitectura	5
R1. Modelo reactivo sobre rxnet	5
R2. Separación por capas	6
R3. Compatibilidad C y Python	6
Requisitos de transporte	7
R4. Transporte abstracto	7
R5. Transporte de referencia	7
Requisitos de persistencia	7
R6. Estado persistente mínimo	7
R7. Escritura segura	7
Requisitos de la máquina de estados de aplicación	8
R8. API de máquina de estados	8
R9. Ejemplo clave-valor	8
Requisitos de observabilidad	8
R10. Observabilidad opcional y externa	8
R11. Eventos estructurados	9
R12. Shell externa	9
Requisitos no funcionales	9
R13. Portabilidad	9
R14. Dependencias	9
R15. Determinismo y testabilidad	10
R16. Calidad de producción	10
Criterios de aceptación	10
Diseño	11
Resumen	11
Modelo de nodo	11

Mapeo sobre fases rxnet	13
Latch inputs	13
Evaluate	13
Commit	13
Deferred actions	14
Dump outputs	14
Tipos de mensaje	14
Log y commit	14
Reconfiguración de clúster	15
Alcance actual de reconfiguración	15
Persistencia	16
Transporte	16
Reloj	17
Observabilidad	17
API pública propuesta	18
Python	18
C	18
Diferencias deliberadas entre C y Python	18
Tareas	21
Documentación	21
Estructura del proyecto	21
Núcleo Raft compartido conceptualmente	21
Python	21
C	22
Verificación	22

List of Figures

Requisitos

Objetivo

Construir una librería Raft sobre `.. /rxnet` en C y en Python, orientada a:

- sistemas empujados distribuidos con restricciones de dependencias,
- aplicaciones de terminal en macOS y Linux,
- integración opcional con herramientas externas de observabilidad,
- ejecución determinista basada en ticks y máquinas de estados.

El resultado debe incluir un ejemplo funcional de base de datos clave-valor distribuida con persistencia.

Alcance funcional

La librería debe implementar el núcleo del algoritmo Raft:

- elección de líder,
- heartbeats y mantenimiento de liderazgo,
- replicación de log,
- confirmación por mayoría,
- aplicación ordenada de entradas a una máquina de estados,
- persistencia de `current_term`, `voted_for` y del log,
- recuperación tras reinicio desde almacenamiento persistente,
- redirección o rechazo explícito de peticiones de cliente si el nodo no es líder.

Requisitos de arquitectura

R1. Modelo reactivo sobre rxnet

Cada nodo Raft debe modelarse como una máquina de estados `rxnet` con, al menos, estos roles:

- FOLLOWER
- CANDIDATE
- LEADER

Las transiciones entre roles deben estar gobernadas por eventos latcheados en el tick actual:

- expiración de timeout de elección,
- recepción de heartbeat o `AppendEntries`,
- recepción de votos,
- descubrimiento de un término superior.

R2. Separación por capas

La solución debe separar claramente:

- núcleo del protocolo Raft,
- transporte,
- persistencia,
- máquina de estados de aplicación,
- observabilidad.

El núcleo no debe depender de una implementación concreta de red, shell o UI.

R3. Compatibilidad C y Python

Debe existir una implementación en C y otra en Python con semántica alineada en:

- tipos de mensaje,
- reglas de transición de término y voto,
- reglas de validación de log,
- semántica de commit,
- interfaz de transporte,
- interfaz de persistencia,
- API de integración con una máquina de estados.

No es obligatorio compartir binarios ni ABI, pero sí el modelo conceptual y el comportamiento observable.

Requisitos de transporte

R4. Transporte abstracto

La librería debe definir una interfaz abstracta de transporte con capacidad para:

- enviar mensajes Raft a un peer,
- recibir todos los mensajes pendientes para un nodo en un tick,
- inyectar peticiones de cliente,
- operar sin asignación dinámica en los caminos críticos del runtime C cuando sea posible.

R5. Transporte de referencia

Debe incluirse al menos un transporte de referencia determinista y portable:

- transporte en memoria para simulación, tests y ejemplo local.

Además, las demos de proceso independiente deben demostrar que el núcleo no depende del bus en memoria:

- C: transporte TCP enchufable con tabla de peers persistente.
- Python: transporte HTTP en la demo externa, implementado con librería estándar.

Los transportes de host deben poder añadirse sin modificar el núcleo Raft.

Requisitos de persistencia

R6. Estado persistente mínimo

Cada nodo debe persistir de forma duradera:

- `current_term`,
- `voted_for`,
- el log Raft,
- el estado aplicado de la máquina de estados del ejemplo clave-valor.

R7. Escritura segura

La persistencia en host debe usar escrituras atómicas o cuasi-atómicas razonables:

- fichero temporal + `rename/replace`,
- formato legible y auditable,
- recuperación robusta ante reinicios limpios.

Para objetivos empotrados, la API debe permitir sustituir esta persistencia por otra adaptada a flash, NVRAM o almacenamiento específico.

Requisitos de la máquina de estados de aplicación

R8. API de máquina de estados

Debe existir una interfaz de aplicación con estas capacidades:

- validar o aceptar comandos serializados,
- aplicar entradas committed en orden,
- consultar estado visible para cliente,
- cargar y guardar estado persistente.

R9. Ejemplo clave-valor

Debe entregarse una máquina de estados de ejemplo con operaciones:

- `set(key, value)`
- `delete(key)`
- `get(key)`

`get` puede resolverse localmente sobre el estado aplicado. `set` y `delete` deben entrar en el log del líder.

Requisitos de observabilidad

R10. Observabilidad opcional y externa

La observabilidad debe ser totalmente opcional. Si no se activa, el núcleo Raft no debe depender de shell, TUI o librerías visuales.

R11. Eventos estructurados

Cuando se active, cada nodo debe poder emitir eventos estructurados externos, al menos para:

- cambio de rol,
- cambio de término,
- voto emitido o recibido,
- recepción y envío de RPCs,
- append y commit de entradas,
- aplicación de comandos a la máquina de estados,
- errores de persistencia o transporte.

R12. Shell externa

Debe incluirse una shell externa básica, cómoda y extensible para Python. Su función mínima será:

- visualizar nodos, roles y términos,
- ver longitud de log y `commit_index`,
- mostrar eventos recientes,
- consultar el estado clave-valor del clúster de ejemplo.

Las demos de proceso independiente pueden incorporar una CLI local como FSM del mismo runtime siempre que el núcleo se mantenga separado de esa UI.

Requisitos no funcionales

R13. Portabilidad

El código C debe compilar al menos en:

- macOS,
- Linux.

La arquitectura debe mantenerse apta para portar a plataformas empujadas.

R14. Dependencias

- C: sin dependencias externas obligatorias aparte de `rxnet` y la libc estándar.
- Python: solo librería estándar más `rxnet` local.
- La shell externa no debe introducir dependencias en el runtime embebido.

R15. Determinismo y testabilidad

La implementación debe poder ejecutarse de forma determinista en tests:

- reloj controlable,
- transporte en memoria,
- inyección explícita de timeouts,
- verificación del estado de clúster y del log.

R16. Calidad de producción

El entregable debe incluir:

- documentación de requisitos, diseño y tareas,
- tests automatizados en C y Python,
- ejemplos ejecutables,
- manejo explícito de errores,
- API pública clara.

Criterios de aceptación

Se considera completado cuando:

1. [docs/requirements.md](#), [docs/design.md](#) y [docs/tasks.md](#) describen de forma coherente el sistema.
2. Existe una librería Python usable y testeada.
3. Existe una librería C usable y compilable.
4. Un clúster de 3 nodos en memoria elige líder y replica operaciones clave-valor.
5. El estado persiste y puede recuperarse tras reinicio.
6. La observabilidad puede activarse o desactivarse sin afectar al núcleo.
7. Existen demos independientes en C y Python que arrancan nodos en procesos separados y permiten reconfiguración de membresía.

Diseño

Resumen

La solución se organiza en cinco capas:

1. `core`: reglas del protocolo Raft y modelado del nodo como FSM `rxnet`.
2. `transport`: entrega de mensajes y peticiones de cliente.
3. `storage`: persistencia de metadatos Raft, log y estado aplicado.
4. `state_machine`: lógica de aplicación; en este proyecto, un almacén clave-valor.
5. `telemetry`: emisión opcional de eventos estructurados para una shell externa.

Las implementaciones C y Python comparten la misma semántica base, aunque no el mismo layout interno. La reconfiguración por `joint consensus` descrita en este documento está implementada en las referencias Python y C.

Modelo de nodo

Cada nodo mantiene:

- `node_id`
- `config_state`
- `peers`
- `current_term`
- `voted_for`
- `log`
- `commit_index`
- `last_applied`
- `role`
- `leader_id`
- `election_deadline_ms`
- `heartbeat_deadline_ms`

- `next_index[peer]`
- `match_index[peer]`

La FSM `rxnet` modela el rol y los eventos principales de Raft:

- `FOLLOWER`
- `CANDIDATE`
- `LEADER`

El resto del estado vive en la estructura de usuario del nodo y se actualiza en callbacks de fase.

Adicionalmente, la implementación Python modela dos FSM secundarias:

- `CompactionFSM`: `IDLE` -> `SNAPSHOT_PENDING` -> `COMPACTING` -> `IDLE`
- `MembershipFSM`: `STABLE` -> `JOINT_PENDING` -> `JOINT` -> `FINALIZING` -> `STABLE`

La membresía ya no es una lista simple. Se representa como una configuración explícita:

- `old_members`
- `new_members` | `None`
- `configuration_index`

Cuando `new_members` is `None`, la configuración es estable. Cuando `new_members` existe, el nodo está en `joint consensus`.

La tabla principal de transición es:

- `FOLLOWER` -- `timeout_expired` --> `CANDIDATE` / `become_candidate`
- `FOLLOWER` -- `has_append_entries` --> `FOLLOWER` / `handle_append_entries`
- `FOLLOWER` -- `has_vote_request` --> `FOLLOWER` / `handle_vote_request`
- `FOLLOWER` -- `has_vote` --> `FOLLOWER` / `ignore_vote`
- `CANDIDATE` -- `timeout_expired` --> `FOLLOWER` / `back_to_follower_due_to_timeout`
- `CANDIDATE` -- `received_majority_votes` --> `LEADER` / `become_leader`
- `CANDIDATE` -- `has_append_entries` --> `FOLLOWER` / `handle_append_entries`
- `CANDIDATE` -- `has_vote_request` --> `CANDIDATE` / `handle_vote_request`
- `CANDIDATE` -- `has_vote` --> `CANDIDATE` / `handle_vote`
- `LEADER` -- `has_append_entries` --> `FOLLOWER` / `handle_append_entries`
- `LEADER` -- `has_vote_request` --> `LEADER` / `ignore_vote_request`
- `LEADER` -- `has_vote` --> `LEADER` / `ignore_vote`

- `LEADER -- time_for_heartbeat --> LEADER / send_heartbeat`

Las colas de eventos se drenan en `latch_inputs` y los guards de la FSM consultan si hay eventos pendientes de cada tipo.

Mapeo sobre fases rxnet

Latch inputs

En `latch_inputs` el nodo:

- lee la hora actual desde un reloj abstracto,
- drena mensajes entrantes del transporte,
- clasifica RPCs y respuestas en colas pendientes,
- detecta si hay timeout de elección o de heartbeat,
- acumula comandos de cliente pendientes,
- calcula flags de transición para la FSM.

En esta fase también se preparan buffers de salida y se marcan escrituras persistentes pendientes.

Evaluate

`rxnet` evalúa la primera transición habilitada según el orden de la tabla anterior. Esto fuerza una semántica clara basada en eventos Raft, no en callbacks laterales.

Commit

La FSM publica el nuevo rol y ejecuta una acción diferida asociada a la transición:

- `become_candidate`: incrementa término, vota por sí mismo y envía `RequestVote`,
- `become_leader`: inicializa índices de replicación y envía heartbeat inmediato,
- `back_to_follower_due_to_timeout`: abandona la candidatura actual y rearma timeout,
- `handle_append_entries`: procesa el RPC del líder,
- `handle_vote_request`: decide concesión de voto,
- `handle_vote`: acumula votos de una elección activa,
- `send_heartbeat`: envía `AppendEntries` vacío,
- `ignore_vote` y `ignore_vote_request`: consumen eventos no relevantes para ese rol.

Deferred actions

Las acciones diferidas se usan solo para cambios de rol y envíos iniciales asociados a la transición. Esto mantiene la separación entre decisión de estado y efectos laterales.

Dump outputs

En `dump_outputs` el nodo:

- persiste metadatos y log si están sucios,
- envía mensajes pendientes,
- avanza la replicación del líder,
- aplica entradas committed a la máquina de estados,
- emite telemetría opcional.

Tipos de mensaje

Se definen cuatro tipos básicos:

- `REQUEST_VOTE`
- `REQUEST_VOTE_RESPONSE`
- `APPEND_ENTRIES`
- `APPEND_ENTRIES_RESPONSE`

Y un tipo auxiliar interno para clientes:

- `CLIENT_COMMAND`

Cada mensaje contiene:

- `term`
- `source`
- `target`
- campos específicos del RPC

`APPEND_ENTRIES` transporta cero o más entradas. Cero entradas equivale a heartbeat.

Log y commit

Cada entrada de log contiene:

- `term`
- `index`
- `leader_id`
- `command`

El líder:

- acepta comandos de cliente,
- los añade al log local,
- replica mediante `AppendEntries`,
- actualiza `commit_index` cuando la entrada está replicada por mayoría en el término actual.

Los followers:

- validan `prev_log_index` y `prev_log_term`,
- corrigen conflictos truncando el sufijo incompatible,
- añaden las nuevas entradas válidas,
- actualizan `commit_index` según `leader_commit`.

Reconfiguración de clúster

Los cambios de membresía no se aplican con una sola entrada. Se modelan como dos entradas de log:

- `cluster.enter_joint(C_new)`
- `cluster.leave_joint(C_new)`

La shell expone operaciones de alto nivel como `addnode` y `rmnode`, pero internamente el líder activa la `MembershipFSM`, que materializa esa secuencia.

Durante `joint consensus`, el cálculo de commit exige doble mayoría:

- mayoría sobre `old_members`
- mayoría sobre `new_members`

La finalización de la transición solo ocurre cuando el líder considera que los miembros de `C_new` están suficientemente puestos al día.

Alcance actual de reconfiguración

La implementación actual cubre:

- representación persistente de configuración estable o joint,
- quorum doble para commit en modo joint,
- transición en dos entradas de log,
- shell operativa para `members`, `addnode` y `rmnode`,
- provisión local de nodos nuevos en el clúster de simulación.

Todavía no cubre:

- `InstallSnapshot` para incorporación acelerada de nodos rezagados,
- restricciones adicionales de elegibilidad de líder durante reconfiguración.

Persistencia

Se usan dos abstracciones:

- `RaftStorage`: estado persistente del protocolo.
- `StateMachineStorage`: persistencia del estado aplicado de la aplicación.

En host, la persistencia de referencia se implementa con ficheros por nodo:

- `meta.json` o `meta.txt`
- `log.jsonl` o `log.txt`
- `kv.json` o `kv.txt`

En `meta.json` se persisten también:

- `old_members`
- `new_members`
- `configuration_index`
- `compaction_threshold`
- metadatos de snapshot

Las escrituras de metadatos y de la máquina de estados se hacen mediante fichero temporal y reemplazo atómico. El log puede reescribirse entero en esta primera versión para simplificar robustez y trazabilidad.

Transporte

La interfaz de transporte ofrece:

- `send(message)`
- `recv_for(node_id) -> mensajes`
- `submit_client_command(node_id, command)`
- `recv_client_commands(node_id)`

La implementación de referencia es un bus en memoria, determinista y sin hilos, útil para:

- tests,
- simulación,
- ejemplo de base de datos distribuida en un solo proceso.

Además del transporte en memoria, el repositorio incluye transportes de demo para ejecución multi-proceso:

- C: `raft_tcp_transport_t`, expuesto en `raft/raft_tcp_transport.h`, con listener TCP, tabla de peers persistente y protocolo de `join/merge`.
- Python: `HttpTransport` dentro de `python/examples/demo_app`, con endpoints REST para mensajes Raft, comandos de cliente y cambios de membresía.

La API deja espacio para transportes futuros sobre UDP, sockets Unix o colas RTOS.

Reloj

El tiempo no se lee directamente del sistema dentro del core. Se abstrae mediante un reloj:

- Python: `Clock.now_ms()`
- C: callback o campo actualizado por el integrador

Esto permite tests deterministas y despliegues empotrados con timers propios.

Observabilidad

En Python, la observabilidad se implementa con un `TelemetrySink` opcional. Si es `NoOp`, no existe coste de dependencia externa.

Los eventos se emiten como estructuras ligeras y pueden serializarse como JSON Lines para ser consumidos por una shell externa.

La shell genérica de Python:

- es un proceso independiente,

- lee eventos desde ficheros JSONL,
- presenta una vista agregada del clúster,
- no forma parte del runtime del nodo.

En C, la observabilidad disponible es el trazado opcional de `rxnet` (`RX_TRACE_ENABLE`) y los ejemplos trazados que exportan `trace.bin`.

API pública propuesta

Python

- `raft_rx.RaftApplication`
- `raft_rx.Command`, `LogEntry`, `Message`, `MessageKind`
- `raft_rx.ManualClock`, `SystemClock`
- `raft_rx.MemoryTransport`
- `raft_rx.JsonFileStorage`
- `raft_rx.JsonlTelemetrySink`, `NullTelemetrySink`
- `raft_rx.NodeConfig`, `RaftNode`, `Role`
- `raft_rx.RaftCluster`
- `raft_rx.RaftShell`

C

- `include/raft/raft.h`
- `include/raft/raft_kv_app.h`
- `include/raft/raft_tcp_transport.h`

`libraft_rx.a` contiene el núcleo, transporte en memoria y persistencia de ficheros. `raft_tcp_transport.c` y `raft_kv_app.c` se compilan junto a la aplicación cuando se necesita TCP o la KV de ejemplo.

Diferencias deliberadas entre C y Python

Python prioriza:

- ergonomía,
- simulación,

- shell externa,
- inspección fácil del estado.

C prioriza:

- API explícita,
- portabilidad a entornos restringidos,
- buffers y capacidades fijas razonables,
- ausencia de dependencias adicionales.

La semántica de protocolo debe mantenerse alineada, aunque la representación interna no sea idéntica.

Tareas

Documentación

- ☒ Redactar requisitos del sistema.
- ☒ Redactar diseño por capas y mapeo sobre [rxnet](#).
- ☒ Definir el alcance de la primera versión productiva.

Estructura del proyecto

- ☒ Crear árbol de código Python.
- ☒ Crear árbol de código C.
- ☒ Añadir README principal y comandos de build/test.

Núcleo Raft compartido conceptualmente

- ☒ Definir tipos de mensaje y entradas de log.
- ☒ Definir semántica de términos, votos y commit.
- ☒ Implementar reglas de validación de log.
- ☒ Implementar recuperación desde almacenamiento persistente.

Python

- ☒ Crear paquete [raft_rx](#).
- ☒ Implementar reloj determinista para tests.
- ☒ Implementar transporte en memoria.
- ☒ Implementar persistencia por ficheros.
- ☒ Implementar [TelemetrySink](#) no-op y JSONL.
- ☒ Implementar máquina de estados clave-valor persistente.

- ☒ Implementar nodo Raft sobre `rxnet.fsm`.
- ☒ Implementar clúster de simulación.
- ☒ Crear ejemplo ejecutable de base de datos distribuida.
- ☒ Crear shell externa para operación e inspección del clúster.
- ☒ Añadir tests de elección, replicación y recuperación.
- ☒ Implementar configuración estable/joint del clúster y FSM secundaria de membresía.
- ☒ Exponer reconfiguración en shell con `members`, `addnode` y `rmnode`.
- ☒ Añadir demo independiente con transporte HTTP, CLI local, `join` y `merge`.

C

- ☒ Crear librería `raft`.
- ☒ Implementar tipos públicos y límites.
- ☒ Implementar transporte en memoria.
- ☒ Implementar persistencia de referencia en ficheros.
- ☒ Añadir trazado opcional vía `rxnet (RX_TRACE_ENABLE)` y ejemplo exportable.
- ☒ Implementar máquina de estados clave-valor persistente.
- ☒ Implementar nodo Raft sobre `rxnet/fsm.h`.
- ☒ Crear ejemplo de clúster de 3 nodos.
- ☒ Añadir tests de elección, replicación y reinicio.
- ☒ Portar reconfiguración `joint consensus` a la librería C.
- ☒ Añadir demo independiente con transporte TCP, CLI local, `join` y `merge`.

Verificación

- ☒ Ejecutar tests Python.
- ☒ Compilar C.
- ☒ Ejecutar tests C.
- ☒ Verificar el ejemplo clave-valor en ambos lenguajes.
- ☒ Actualizar la documentación con el estado final del entregable.